

PRACTICAL - 06**Part A — Practice Programs (Hour 1)**

PROBLEM1	1. Remote control: an interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda.
CODE	<pre> interface Switchable { void on(); void off(); default void toggle() { on(); off(); } } class Fan implements Switchable { public void on() { System.out.println("Fan is ON"); } public void off() { System.out.println("Fan is OFF"); } } class Light implements Switchable { public void on() { System.out.println("Light is ON"); } public void off() { System.out.println("Light is OFF"); } } @FunctionalInterface interface Permission { boolean maySwitchOn(String device, int hour); } </pre>

```
public class RemoteControl {

    public static void main(String[] args) {

        Fan fan = new Fan();
        Light light = new Light();

        Switchable[] devices = {fan, light};

        System.out.println("Toggling devices:");

        for (Switchable device : devices) {
            device.toggle();
        }

        Permission p1 = new Permission() {
            public boolean maySwitchOn(String device, int hour) {
                return hour >= 6 && hour <= 22;
            }
        };

        System.out.println("Fan at 10: " + p1.maySwitchOn("Fan", 10));
        System.out.println("Light at 2: " + p1.maySwitchOn("Light", 2));

        Permission p2 = (device, hour) -> hour >= 6 && hour <= 22;

        System.out.println("\nUsing lambda:");
        System.out.println("Fan at 8: " + p2.maySwitchOn("Fan", 8));
        System.out.println("Light at 3: " + p2.maySwitchOn("Light", 3));
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP> cd "PRACTICAL 6"
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 6> javac RemoteControl.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 6> java RemoteControl.java
Toggling devices:
Fan is ON
Fan is OFF
Light is ON
Light is OFF
Fan at 10: true
Light at 2: false

Using lambda:
Fan at 8: true
Light at 3: false
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 6> █
```

PROBLEM2	2. Notification senders: a functional interface Notifier with send (String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent.
CODE	<pre>@FunctionalInterface interface Notifier { void send(String message); } interface Urgent { } class Email implements Urgent { String name; Email(String name) { this.name = name; } } class SMS { String name; SMS(String name) { this.name = name; } } public class NotificationSender { public static void main(String[] args) { Email email = new Email("Email"); SMS sms = new SMS("SMS"); Notifier email_sender = message -> System.out.println("Email: " + message); Notifier sms_sender = message -> System.out.println("SMS: " + message); Notifier[] senders = {email_sender, sms_sender}; String message = "Meeting at 5 PM"; for (Notifier sender : senders) { sender.send(message); } } }</pre>

	<pre> if (sender == email_sender && email instanceof Urgent) { sender.send(message); } } } } </pre>
OUTPUT	<pre> PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 6> javac NotificationSender.java PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 6> java NotificationSender Email: Meeting at 5 PM Email: Meeting at 5 PM SMS: Meeting at 5 PM PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 6> </pre>
PROBLEM3	3. (For Advanced Learners – home) Build a small discount engine where each rule is a lambda the user picks at runtime and applies to a list of prices; organise it into packages and ship it as a runnable JAR.
CODE	<u>DiscountApp.java</u> <pre> package discount; import java.util.Scanner; public class DiscountApp { public static void main(String[] args) { Scanner sc = new Scanner(System.in); double[] prices = {500, 1000, 1500, 2000}; System.out.println("Choose discount:"); System.out.println("1. 10% discount"); System.out.println("2. 20% discount"); System.out.println("3. 50% discount"); System.out.print("Enter choice: "); int choice = sc.nextInt(); DiscountRule rule; if (choice == 1) { rule = price -> price - (price * 0.10); } } </pre>

```
else if (choice == 2) {  
    rule = price -> price - (price * 0.20);  
}  
else if (choice == 3) {  
    rule = price -> price - (price * 0.50);  
}  
else {  
    System.out.println("Invalid choice");  
    sc.close();  
    return;  
}  
  
System.out.println("\nPrices after discount:");  
  
for (double price : prices) {  
    double final_price = rule.apply(price);  
    System.out.println(price + " -> " + final_price);  
}  
  
sc.close();  
}
```

DiscountRule.java

```
package discount;  
  
@FunctionalInterface  
public interface DiscountRule {  
    double apply(double price);  
}
```

OUTPUT

```
Problems 36 Output Debug Console Terminal Ports  
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 6> javac -d out src\discount\*.java  
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 6> java -cp out discount.DiscountApp  
Choose discount:  
1. 10% discount  
2. 20% discount  
3. 50% discount  
Enter choice: 3  
  
Prices after discount:  
500.0 -> 250.0  
1000.0 -> 500.0  
1500.0 -> 750.0  
2000.0 -> 1000.0  
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 6> |
```